TY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT 4 – CLASSIFICATION MODELS

1. Student Details

Student Name	Revati Bhosale
PRN	202401040048
Branch	Computer Engineering
Division	C
Batch	C2
Roll No.	173
Date of Submission	22/10/2026

2. Practical Details

Practical No.	4
Practical Title	Classification Models
Date Performed	22/10/2026
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

Develop classification models using Support Vector Machine (SVM), Decision Tree, K-Nearest Neighbours (KNN) and Naïve Bayes. Evaluate the performance of each model using relevant metrics – accuracy, precision, recall, F1-score and loss – and summarise the findings of the comparative analysis to discuss the suitability of each algorithm for various classification tasks.

The models are trained on the Breast Cancer Wisconsin (Diagnostic) dataset, a binary classification problem in which a tumour must be classified as malignant or benign from 30 numerical features computed from digitised images of a fine-needle aspirate.

4. Objectives

1. To implement SVM, Decision Tree, KNN and Naïve Bayes classifiers on the same dataset using an identical train/test split.
2. To evaluate every model using accuracy, precision, recall, F1-score, log loss and the confusion matrix.
3. To compare the models and discuss which algorithm is suitable for which type of classification task.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Explain the working principle of SVM, Decision Tree, KNN and Naïve Bayes classifiers.
2. Train and test classification models in Python using scikit-learn, including feature scaling where it is required.
3. Compute and interpret accuracy, precision, recall, F1-score, loss and the confusion matrix.
4. Compare algorithms on the basis of performance, interpretability, speed and data characteristics, and select a suitable one for a given task.

6. Theory / Concept

Classification: a supervised learning task in which a model learns a mapping from input features to a discrete class label using labelled training data, and then predicts the label of unseen data.

Support Vector Machine (SVM): finds the hyperplane that separates the classes with the maximum margin. With the RBF kernel, data is implicitly mapped to a higher-dimensional space so that non-linearly separable classes can

be separated. The parameter C controls the trade-off between a wide margin and training errors. SVM is distance-based, so features must be scaled.

Decision Tree: a tree of if-else rules. At each node the feature and threshold giving the best split (lowest Gini impurity, or highest information gain) are chosen, and the data is divided until a leaf is reached. It is easy to interpret and needs no feature scaling, but an unrestricted tree overfits easily, so the depth is limited (max_depth = 5).

K-Nearest Neighbours (KNN): a lazy, instance-based learner with no explicit training phase. A new sample is assigned the majority class among its k closest training samples (Euclidean distance). It is sensitive to feature scale and slow at prediction time on large datasets.

Naïve Bayes: a probabilistic classifier based on Bayes' theorem, $P(C|x) \propto P(C) \cdot \prod P(x_i|C)$, which assumes that features are conditionally independent given the class. Gaussian Naïve Bayes models each continuous feature with a normal distribution per class. It is extremely fast and works with little data.

Evaluation metrics (positive class = benign)

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1-score  = 2 × Precision × Recall / (Precision + Recall)
Log loss  = −(1/N) Σ [ y·log(p) + (1 − y)·log(1 − p) ]
```

Log loss (cross-entropy loss) measures the quality of the predicted probabilities: confident wrong predictions are penalised heavily, so a lower value is better. The confusion matrix shows the counts of true/false positives and negatives.

7. Algorithm / Pseudocode

Step-by-step algorithm for the proposed solution:

```
START

Step 1: Import the libraries and load the Breast Cancer dataset (X = features, y =
labels).
Step 2: Split the data into 80% training and 20% testing sets (stratified, random_state =
42).
Step 3: Create the four models: SVM (RBF), Decision Tree, KNN (k = 5), Gaussian Naive
Bayes.
        Apply StandardScaler before SVM and KNN, since they are distance based.
Step 4: FOR each model:
            Train the model on the training set.
            Predict labels and class probabilities for the test set.
            Compute accuracy, precision, recall, F1-score, log loss and the confusion
matrix.
Step 5: Tabulate the metrics and plot the confusion matrices and the metric comparison
chart.
Step 6: Compare the models and discuss the suitability of each algorithm.

STOP
```

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	Python 3 – implementation of all four classification models
IDE / Platform	Jupyter Notebook / VS Code – writing, running and

	visualising the code
Libraries / Packages	scikit-learn (models, metrics), NumPy (numerical arrays), Matplotlib (graphs)
Dataset / Other Resources	Breast Cancer Wisconsin (Diagnostic) dataset, built into scikit-learn (569 samples, 30 features)

09. Implementation / GitHub Repository

Particular	**Details**
GitHub Repository URL	**https://github.com/Revati-2920/Classification-Models.git**
Repository Name	**Classification Models**
Branch Used	**main / master / Other**
Commit ID / Latest Commit	**-**
Date of Last Commit	**22/10/2026**

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1: Load dataset	load_breast_cancer()	569 samples, 30 features, 2 classes	569 × 30; 212 malignant, 357 benign	Pass
TC2: Train/test split	test_size = 0.2, stratified	≈ 455 train / 114 test samples	455 train / 114 test	Pass
TC3: SVM	114 test samples	Accuracy > 90%	Accuracy = 98.25%	Pass
TC4: Decision Tree	114 test samples	Accuracy > 90%	Accuracy = 92.11%	Pass
TC5: KNN	114 test samples	Accuracy > 90%	Accuracy = 95.61%	Pass
TC6: Naive Bayes	114 test samples	Accuracy > 90%	Accuracy = 93.86%	Pass

11. Results / Observations

The dataset has 569 samples (212 malignant, 357 benign) and 30 features. It was split into 455 training and 114 test samples. All four models were trained on the same training set and evaluated on the same test set. The results are given below.

Table 1: Performance of the classification models on the test set

Model	Accuracy	Precision	Recall	F1-score	Log loss
SVM	0.9825	0.9861	0.9861	0.9861	0.0860
Decision Tree	0.9211	0.9565	0.9167	0.9362	1.8916
KNN	0.9561	0.9589	0.9722	0.9655	0.3941
Naive Bayes	0.9386	0.9452	0.9583	0.9517	0.3713

Table 2: Confusion matrices (rows = actual, columns = predicted; positive class = benign)

Model	TN	FP	FN	TP	Total errors
SVM	41	1	1	71	2
Decision Tree	39	3	6	66	9
KNN	39	3	2	70	5
Naive Bayes	38	4	3	69	7

Output of the program:

```
SVM           Acc=0.9825  Prec=0.9861  Rec=0.9861  F1=0.9861  Loss=0.0860
Decision Tree Acc=0.9211  Prec=0.9565  Rec=0.9167  F1=0.9362  Loss=1.8916
KNN           Acc=0.9561  Prec=0.9589  Rec=0.9722  F1=0.9655  Loss=0.3941
Naive Bayes   Acc=0.9386  Prec=0.9452  Rec=0.9583  F1=0.9517  Loss=0.3713
```

Screenshots / Graphs / Output:

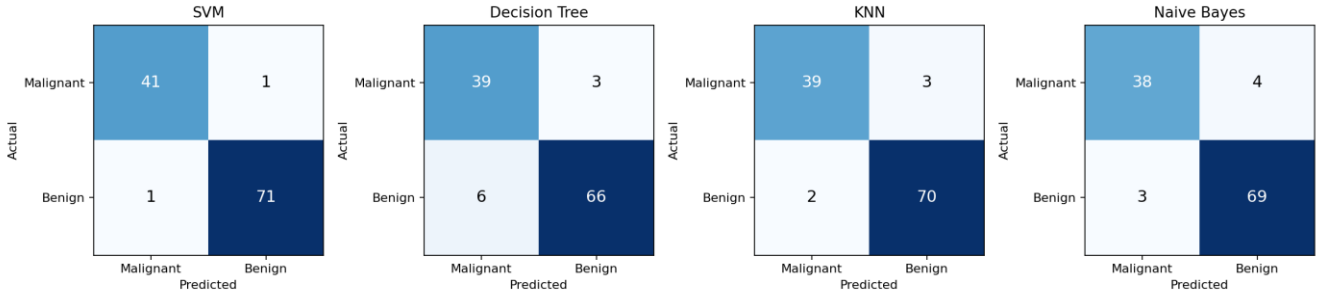


Figure 1: Confusion matrices of SVM, Decision Tree, KNN and Naive Bayes

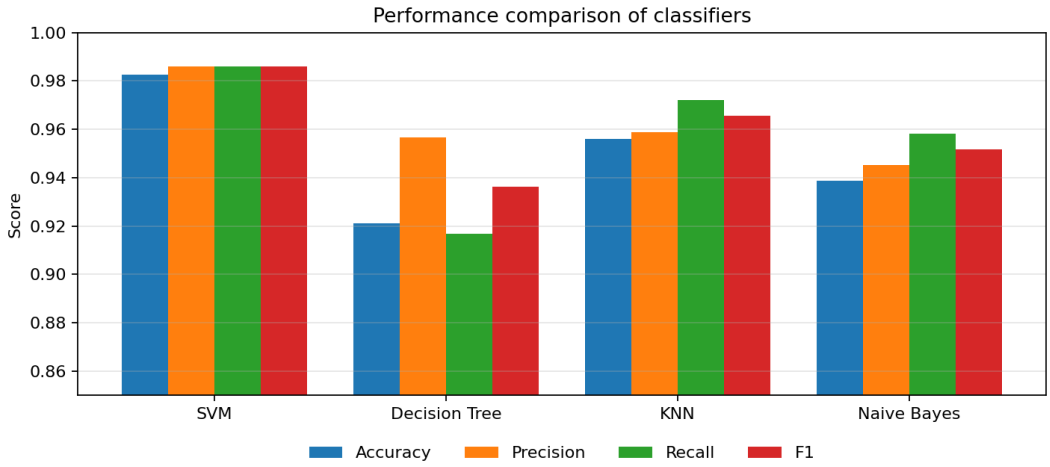


Figure 2: Comparison of accuracy, precision, recall and F1-score

12. Performance Evaluation / Analysis

1. **SVM** achieved the best result on every metric: accuracy 98.25%, F1-score 98.61% and the lowest log loss (0.0860). It made only 2 errors out of 114 test samples (1 false positive, 1 false negative). Standardised features and the RBF kernel suit this dataset, whose classes are almost separable in a high-dimensional space.
2. **KNN** came second with accuracy 95.61% and recall 97.22% (only 2 benign cases missed), after feature scaling. Its log loss (0.3941) is higher than SVM's because with k = 5 the predicted probabilities can only take the values 0, 0.2, 0.4 … 1.
3. **Naive Bayes** gave accuracy 93.86% and F1-score 95.17% despite being the simplest model. Its log loss (0.3713) is the second lowest among the four, but its independence assumption is violated here because many features (radius, perimeter, area) are strongly correlated, which limits its accuracy.
4. **Decision Tree** was the weakest: accuracy 92.11%, recall 91.67% and a very high log loss of 1.8916. A tree outputs the class proportion of a leaf as its probability, which is often exactly 0 or 1; when such a prediction is wrong, the log loss penalty is very large. A single tree also has high variance, so an ensemble (Random Forest) would improve it.

Because the dataset is a medical one, recall of the benign class and the number of false negatives/false positives matter alongside accuracy; an F1-score close to 1 shows that precision and recall are balanced. The results are from a single 80/20 hold-out split; k-fold cross-validation would give a more reliable estimate.

Comparative suitability of the algorithms

Algorithm	Strengths	Limitations	Suitable for
SVM	High accuracy; effective in high-dimensional spaces; kernels handle non-linear boundaries; good generalisation.	Needs feature scaling; slow on very large data; hard to interpret; sensitive to C and gamma.	Small/medium datasets with many features: medical diagnosis, text and image classification.
Decision Tree	Easy to interpret and visualise; no scaling needed; handles mixed feature types; fast prediction.	Overfits easily; unstable to small data changes; poor probability estimates.	Tasks needing explainable rules: credit approval, rule-based decisions; base learner for ensembles.
KNN	Simple; no training phase; naturally non-linear; adapts easily to new data.	Slow prediction on large data; needs scaling; affected by irrelevant features and the choice of k.	Small datasets with few features: recommendation, pattern recognition, baseline models.
Naive Bayes	Very fast to train and predict; works with small data; handles many features; gives probabilities.	Independence assumption rarely true; lower accuracy with correlated features.	Text classification, spam filtering, sentiment analysis, real-time prediction.

13. Conclusion

SVM, Decision Tree, KNN and Naïve Bayes classifiers were implemented on the Breast Cancer Wisconsin dataset and compared using accuracy, precision, recall, F1-score, log loss and confusion matrices. SVM performed best (accuracy 98.25%, F1-score 98.61%, loss 0.0860), followed by KNN (95.61%), Naïve Bayes (93.86%) and Decision Tree (92.11%). It was learnt that no single algorithm is best for every task: the choice depends on dataset size, number and correlation of features, need for interpretability, training/prediction speed and the need for reliable probabilities. It was also observed that scaling is essential for SVM and KNN, and that accuracy alone is not enough – loss and recall give a fuller picture of model quality.

14. Assignment Questions

Q1. Compare the four classifiers on the basis of the evaluation metrics. Which model performed best and why?

Answer: SVM performed best (accuracy 98.25%, precision/recall/F1 = 0.9861, log loss 0.0860), then KNN, Naïve Bayes and Decision Tree. SVM finds the maximum-margin boundary in a scaled feature space and generalises well on this high-dimensional, nearly separable dataset. The Decision Tree overfits and gives over-confident probabilities, hence its high loss.

Q2. Discuss the suitability of each algorithm for different classification tasks.

Answer: SVM suits small/medium, high-dimensional data such as medical, image and text features. Decision Trees suit problems that need interpretable rules and mixed data types. KNN suits small, low-dimensional datasets and quick baselines. Naïve Bayes suits text classification, spam filtering and real-time systems where speed and small training data matter. (See the comparison table in Section 12.)

Q3. Why is feature scaling required for SVM and KNN but not for Decision Trees and Naïve Bayes?

Answer: SVM and KNN depend on distances between samples, so a feature with a large range (such as area) would dominate features with small ranges. Decision Trees only compare one feature with a threshold at a time, and Gaussian Naïve Bayes models each feature separately, so neither is affected by feature scale.

15. Implementation / Source Code

Complete, commented source code:

```
"""Practical 4: Classification Models (SVM, Decision Tree, KNN, Naive Bayes)
Dataset: Breast Cancer Wisconsin (Diagnostic) - built into scikit-learn."""
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, log_loss, confusion_matrix)

data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

models = {
    "SVM": Pipeline([("sc", StandardScaler()),
                     ("m", SVC(kernel="rbf", C=1.0, probability=True, random_state=42))]),
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "KNN": Pipeline([("sc", StandardScaler()),
                     ("m", KNeighborsClassifier(n_neighbors=5))]),
    "Naive Bayes": GaussianNB(),
}

results, cms = {}, {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    proba = np.clip(model.predict_proba(X_test), 1e-15, 1 - 1e-15)
    results[name] = {
        "accuracy": accuracy_score(y_test, pred),
        "precision": precision_score(y_test, pred),
        "recall": recall_score(y_test, pred),
        "f1": f1_score(y_test, pred),
        "loss": log_loss(y_test, proba),
    }
    cms[name] = confusion_matrix(y_test, pred).tolist()
    print(name, {k: round(v, 4) for k, v in results[name].items()}, cms[name])

json.dump({"results": results, "cms": cms, "n_train": len(y_train), "n_test": len(y_test),
           "n_features": X.shape[1], "counts": np.bincount(y).tolist()},
          open("results.json", "w"), indent=1)

# Confusion matrices
fig, axes = plt.subplots(1, 4, figsize=(15, 3.6))
for ax, (name, cm) in zip(axes, cms.items()):
    cm = np.array(cm)
    ax.imshow(cm, cmap="Blues")
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center",
                    color="white" if cm[i, j] > cm.max() / 2 else "black", fontsize=13)
    ax.set_title(name); ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
```

```
    ax.set_xticklabels(["Malignant", "Benign"]); ax.set_yticklabels(["Malignant", "Benign"])
    ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
plt.tight_layout(); plt.savefig("confusion_matrices.png", dpi=160); plt.close()

# Metric comparison
names = list(results); metrics = ["accuracy", "precision", "recall", "f1"]
w = 0.2; x = np.arange(len(names))
fig, ax = plt.subplots(figsize=(9, 4.2))
for i, m in enumerate(metrics):
    ax.bar(x + i * w - 1.5 * w, [results[n][m] for n in names], w, label=m.capitalize())
ax.set_xticks(x); ax.set_xticklabels(names); ax.set_ylim(0.85, 1.0)
ax.set_ylabel("Score"); ax.set_title("Performance comparison of classifiers")
ax.legend(ncol=4, loc="upper center", bbox_to_anchor=(0.5, -0.1), frameon=False); ax.grid(axis="y",
alpha=.3)
plt.tight_layout(); plt.savefig("metrics_comparison.png", dpi=160); plt.close()
```

16. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Revati Bhosale	202401040048	